

**LAPORAN PRAKTIKUM ALGORITMA DAN STRUKTUR
DASAR**

JOBSHEET 12 : DOUBLE LINKED LIST



Oleh :

SALSABILA KEISHA AYU SETIADI

254107020048

Kelas TI-1C

PROGRAM STUDI D-IV TEKNIK INFORMATIKA

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI MALANG

A. Praktikum 1

1. Kode Program Class Mahasiswa26 :

```
1 package Minggu12;
2
3 public class Mahasiswa26 {
4     String nim, nama, kelas;
5     double ipk;
6
7     Mahasiswa26(String nm, String nma, String kls, double ip) {
8         nim = nm;
9         nama = nma;
10        kelas = kls;
11        ipk = ip;
12    }
13
14    void tampil() {
15        System.out.println(
16            "NIM      : " + nim +
17            "\nNama    : " + nama +
18            "\nKelas  : " + kelas +
19            "\nIPK     : " + ipk
20        );
21    }
22 }
```

2. Kode Program Class Node26:

```
1 package Minggu12;
2
3 public class Node26 {
4     Mahasiswa26 data;
5     Node26 prev;
6     Node26 next;
7
8     Node26(Mahasiswa26 data) {
9         this.data = data;
10        this.prev = null;
11        this.next = null;
12    }
13 }
14
```

3. Kode Pogram Class DoubleLinkedList26 :

```
Minggu12 > DoubleLinkedList26.java > DoubleLinkedList26 > removeLast()
1  package Minggu12;
2
3  public class DoubleLinkedList26 {
4      Node26 head;
5      Node26 tail;
6
7      DoubleLinkedList26() {
8          head = null;
9          tail = null;
10     }
11
12     boolean isEmpty() {
13         return head == null;
14     }
15
16     void addFirst(Mahasiswa26 data) {
17         Node26 newNode = new Node26(data);
18         if (isEmpty()) {
19             head = tail = newNode;
20         } else {
21             newNode.next = head;
22             head.prev = newNode;
23             head = newNode;
24         }
25     }
26
27     void addLast(Mahasiswa26 data) {
28         Node26 newNode = new Node26(data);
29         if (isEmpty()) {
30             head = tail = newNode;
31         } else {
32             tail.next = newNode;
33             newNode.prev = tail;
34             tail = newNode;
35         }
36     }
37
38     void insertAfter(String keyNim, Mahasiswa26 data) {
39         Node26 current = head;
40         while (current != null && !current.data.nim.equals(keyNim)) {
41             current = current.next;
42         }
43         if (current == null) {
44             System.out.println("Data dengan NIM " + keyNim + " tidak ditemukan.");
45             return;
46         }
47
48         Node26 newNode = new Node26(data);
49
50         //Jika current tail, maka akan ditambahkan di akhir
51         if (current == tail) {
52             newNode.prev = current;
53             current.next = newNode;
54             tail = newNode;
55         } else {
56             newNode.prev = current;
57             newNode.next = current.next;
58             current.next.prev = newNode;
59             current.next = newNode;
60         }
61         System.out.println("Data berhasil disisipkan setelah NIM " + keyNim);
62     }
63
64     void print() {
65         if (isEmpty()) {
66             System.out.println(x: "Linked List masih kosong.");
67             return;
68         }
69
70         Node26 current = head;
71         while (current != null) {
72             current.data.tampil();
73             current = current.next;
74         }
75     }
76 }
```

4. Kode Program Main DoubleLinkedListMain26:

```
Minggu12 > DoubleLinkedListMain26.java > DoubleLinkedListMain26 > main(String[])
1 package Minggu12;
2 import java.util.Scanner;
3 public class DoubleLinkedListMain26 {
4
5     public static Mahasiswa26 inputMahasiswa(Scanner sc) {
6         System.out.print(s: "Masukkan NIM      : ");
7         String nim = sc.nextLine();
8         System.out.print(s: "Masukkan Nama    : ");
9         String nama = sc.nextLine();
10        System.out.print(s: "Masukkan Kelas   : ");
11        String kelas = sc.nextLine();
12        System.out.print(s: "Masukkan IPK    : ");
13        double ipk = sc.nextDouble();
14        sc.nextLine();
15        return new Mahasiswa26(nim, nama, kelas, ipk);
16    }
17
18    Run | Debug
19    public static void main(String[] args) {
20        Scanner sc = new Scanner(System.in);
21        DoubleLinkedList26 list = new DoubleLinkedList26();
22        int pilihan;
23
24        do {
25            System.out.println(x: "\n==== MENU DOUBLE LINKED LIST ====");
26            System.out.println(x: "1. Tambah data di Awal");
27            System.out.println(x: "2. Tambah data di Akhir");
28            System.out.println(x: "3. Sisipkan data di tengah (Setelah NIM)");
29            System.out.println(x: "4. Hapus data di Awal");
30            System.out.println(x: "5. Hapus data di Akhir");
31            System.out.println(x: "6. Tampilkan Data (Mulai Awal)");
32            System.out.println(x: "7. Tampilkan Data (Mulai Akhir)"); //Modifikasi penambahan method
33            System.out.println(x: "0. Keluar");
34            System.out.print(s: "Pilih Menu : ");
35            pilihan = sc.nextInt();
36            sc.nextLine();
37
38            switch (pilihan) {
39                case 1:
40                    Mahasiswa26 mhsAwal = inputMahasiswa(sc);
41                    list.addFirst(mhsAwal);
42                    break;
43                case 2:
44                    Mahasiswa26 mhsAkhir = inputMahasiswa(sc);
45                    list.addLast(mhsAkhir);
46                    break;
47                case 3:
48                    System.out.print(s: "Masukkan NIM yang dicari : ");
49                    String keyNim = sc.nextLine();
50                    System.out.println(x: "Masukkan data baru : ");
51                    Mahasiswa26 dataBaru = inputMahasiswa(sc);
52                    list.insertAfter(keyNim, dataBaru);
53                    break;
54                case 4:
55                    list.removeFirst();
56                    break;
57                case 5:
58                    list.removeLast();
59                    break;
60                case 6:
61                    list.print();
62                    break;
63                case 7: // Modifikasi penambahan method
64                    list.printReverse();
65                    break;
66                case 0:
67                    System.out.println(x: "Program Selesai.");
68                    break;
69                default:
70                    System.out.println(x: "Pilihan tidak valid.");
71            }
72        } while (pilihan != 0);
73        sc.close();
74    }
75}
```

5. Hasil Output:

```
==== MENU DOUBLE LINKED LIST ====
1. Tambah data di Awal
2. Tambah data di Akhir
3. Sisipkan data di tengah (Setelah NIM)
4. Hapus data di Awal
5. Hapus data di Akhir
6. Tampilkan Data
0. Keluar
Pilih Menu : 2
Masukkan NIM      : 123005
Masukkan Nama     : Harry
Masukkan Kelas    : 1A
Masukkan IPK      : 3.76

==== MENU DOUBLE LINKED LIST ====
1. Tambah data di Awal
2. Tambah data di Akhir
3. Sisipkan data di tengah (Setelah NIM)
4. Hapus data di Awal
5. Hapus data di Akhir
6. Tampilkan Data
0. Keluar
Pilih Menu : 3
Masukkan NIM yang dicari : 123005
Masukkan data baru :
Masukkan NIM      : 123010
Masukkan Nama     : Potter
Masukkan Kelas    : 1B
Masukkan IPK      : 3.55
Data berhasil disisipkan setelah NIM 123005

==== MENU DOUBLE LINKED LIST ====
1. Tambah data di Awal
2. Tambah data di Akhir
3. Sisipkan data di tengah (Setelah NIM)
4. Hapus data di Awal
5. Hapus data di Akhir
6. Tampilkan Data
0. Keluar
Pilih Menu : 6
NIM      : 123005
Nama     : Harry
Kelas   : 1A
IPK      : 3.76
NIM      : 123010
Nama     : Potter
Kelas   : 1B
IPK      : 3.55

==== MENU DOUBLE LINKED LIST ====
1. Tambah data di Awal
2. Tambah data di Akhir
3. Sisipkan data di tengah (Setelah NIM)
4. Hapus data di Awal
5. Hapus data di Akhir
6. Tampilkan Data
0. Keluar
Pilih Menu : 0
Program Selesai.
```

6. Jawaban dari pertanyaan :

- 1.) Perbedaan antara Single dan Double Linked List adalah jika Single Linked List hanya memiliki satu pointer yaitu next. Sementara pada Double Linked List memiliki dua pointer yaitu next dan prev. Selain itu, Single Linked List hanya bisa melakukan penelusuran data secara maju dan tidak bisa mundur. Sementara pada Double Linked List, bisa melakukan pencarian melalui dua arah, dari head dan juga tail.
- 2.) Fungsi dari atribut next adalah untuk menunjuk penyimpanan node setelahnya. Atribut next juga digunakan untuk menelusuri data secara maju. Selain itu, terdapat atribut prev yang berfungsi untuk menunjuk atau menyimpan alamat node sebelumnya. Atribut prev juga digunakan untuk menelusuri data secara mundur (dimulai dari tail).

- 3.) Konstruktor tersebut berisi bahwa head dan tail berisi null, yang berarti pada awal data dideklarasikan, nilai head dan tail masih kosong yang juga berarti data masih kosong.
- 4.) Karena Linked List hanya berisi satu data saja, karena hanya berisi satu data saja maka otomatis head dan tail berada di posisi yang sama.
- 5.) Modifikasi yang dilakukan :

```
void print() {  
    if (isEmpty()) {  
        System.out.println(x: "Linked List masih kosong.");  
        return;  
    }  
}
```

Hasil Output:

```
==== MENU DOUBLE LINKED LIST ====  
1. Tambah data di Awal  
2. Tambah data di Akhir  
3. Sisipkan data di tengah (Setelah NIM)  
4. Hapus data di Awal  
5. Hapus data di Akhir  
6. Tampilkan Data  
0. Keluar  
Pilih Menu : 6  
Linked List masih kosong.
```

- 6.) Modifikasi yang dilakukan ;
 - a. Penambahan pada Class DoubleLinkedList26:

```
77 // Modifikasi penambahan method  
78 void printReverse() {  
79     if (isEmpty()) {  
80         System.out.println(x: "Linked List masih kosong.");  
81         return;  
82     }  
83  
84     Node26 current = tail;  
85     while (current != null) {  
86         current.data.tampil();  
87         current = current.prev;  
88     }  
89 }  
90
```

b. Hasil Output

```
==== MENU DOUBLE LINKED LIST ====
1. Tambah data di Awal
2. Tambah data di Akhir
3. Sisipkan data di tengah (Setelah NIM)
4. Hapus data di Awal
5. Hapus data di Akhir
6. Tampilkan Data (Mulai Awal)
7. Tampilkan Data (Mulai Akhir)
0. Keluar
Pilih Menu : 1
Masukkan NIM      : 123
Masukkan Nama     : nanami
Masukkan Kelas    : 1C
Masukkan IPK      : 4.0
```

```
==== MENU DOUBLE LINKED LIST ====
1. Tambah data di Awal
2. Tambah data di Akhir
3. Sisipkan data di tengah (Setelah NIM)
4. Hapus data di Awal
5. Hapus data di Akhir
6. Tampilkan Data (Mulai Awal)
7. Tampilkan Data (Mulai Akhir)
0. Keluar
Pilih Menu : 2
Masukkan NIM      : 124
Masukkan Nama     : kei
Masukkan Kelas    : 1C
Masukkan IPK      : 4.0
```

```
==== MENU DOUBLE LINKED LIST ====
1. Tambah data di Awal
2. Tambah data di Akhir
3. Sisipkan data di tengah (Setelah NIM)
4. Hapus data di Awal
5. Hapus data di Akhir
6. Tampilkan Data (Mulai Awal)
7. Tampilkan Data (Mulai Akhir)
0. Keluar
Pilih Menu : 3
Masukkan NIM yang dicari : 123
Masukkan data baru :
Masukkan NIM      : 126
Masukkan Nama     : Aksan
Masukkan Kelas    : 1C
Masukkan IPK      : 3.7
Data berhasil disisipkan setelah NIM 123
```

```
==== MENU DOUBLE LINKED LIST ====
1. Tambah data di Awal
2. Tambah data di Akhir
3. Sisipkan data di tengah (Setelah NIM)
4. Hapus data di Awal
5. Hapus data di Akhir
6. Tampilkan Data (Mulai Awal)
7. Tampilkan Data (Mulai Akhir)
0. Keluar
Pilih Menu : 7
NIM      : 124
Nama     : kei
Kelas   : 1C
IPK      : 4.0
NIM      : 126
Nama     : Aksan
Kelas   : 1C
IPK      : 3.7
NIM      : 123
Nama     : nanami
Kelas   : 1C
IPK      : 4.0
```

B. Praktikum 2

1. Penambahan pada Class DoubleLinkedList26:

```
91     void removeFirst() {
92         if (isEmpty()) {
93             System.out.println(x: "Linked List kosong.");
94             return;
95         }
96
97         // Modifikasi pertanyaan 2
98         System.out.println(x: "Data berhasil dihapus: ");
99         head.data.tampil();
100
101         if (head == tail) {
102             head = tail = null;
103         } else {
104             head = head.next;
105             head.prev = null;
106         }
107     }
108
109     void removeLast() {
110         if (isEmpty()) {
111             System.out.println(x: "Linked List Kosong.");
112             return;
113         }
114
115         // Modifikasi pertanyaan 2
116         System.out.println(x: "Data berhasil dihapus: ");
117         tail.data.tampil();
118
119         if (head == tail) {
120             head = tail = null;
121         } else {
122             tail = tail.prev;
123             tail.next = null;
124         }
125     }
126 }
```

2. Output dari Kode:

```
==== MENU DOUBLE LINKED LIST ====
1. Tambah data di Awal
2. Tambah data di Akhir
3. Sisipkan data di tengah (Setelah NIM)
4. Hapus data di Awal
5. Hapus data di Akhir
6. Tampilkan Data (Mulai Awal)
7. Tampilkan Data (Mulai Akhir)
0. Keluar
Pilih Menu : 5

==== MENU DOUBLE LINKED LIST ====
1. Tambah data di Awal
2. Tambah data di Akhir
3. Sisipkan data di tengah (Setelah NIM)
4. Hapus data di Awal
5. Hapus data di Akhir
6. Tampilkan Data (Mulai Awal)
7. Tampilkan Data (Mulai Akhir)
0. Keluar
Pilih Menu : 6
NIM      : 123005
Nama     : Harry
Kelas   : 1A
IPK      : 3.76
```


3. Jawaban dari pertanyaan :

1.) Potongan kode tersebut berfungsi untuk memutus tali dengan node paling awal. Posisi head dipindah ke posisi head.next atau node kedua. Lalu, posisi head.prev (yang berarti prev dari node kedua) dijadikan null/kosong sehingga node tidak berhubungan dengan node manapun.

2.) Modifikasi yang dilakukan:

a. Penambahan potongan kode Program pada method removeFirst dan removeLast

```
void removeFirst() {  
    if (isEmpty()) {  
        System.out.println(x: "Linked List kosong.");  
        return;  
    }  
  
    // Modifikasi pertanyaan 2  
    System.out.println(x: "Data berhasil dihapus: ");  
    head.data.tampil();  
}
```

```
void removeLast() {  
    if (isEmpty()) {  
        System.out.println(x: "Linked List Kosong.");  
        return;  
    }  
  
    // Modifikasi pertanyaan 2  
    System.out.println(x: "Data berhasil dihapus: ");  
    tail.data.tampil();  
}
```

b. Hasil Output :

```
==== MENU DOUBLE LINKED LIST ====  
1. Tambah data di Awal  
2. Tambah data di Akhir  
3. Sisipkan data di tengah (Setelah NIM)  
4. Hapus data di Awal  
5. Hapus data di Akhir  
6. Tampilkan Data (Mulai Awal)  
7. Tampilkan Data (Mulai Akhir)  
0. Keluar  
Pilih Menu : 5  
Data berhasil dihapus:  
NIM      : 123010  
Nama     : Potter  
Kelas   : 1B  
IPK      : 3.5
```